Choosing among four mathematical solutions

DASE7506 Mini-Project 2 | 31 October 2026

1. Task and approach

The submitted program chooses a correct final answer on **399 of 467 test questions (85.44%)**. It combines a linear verifier with two small neural verifiers. Complete test prediction and scoring took 3.14 seconds on the local CPU, using 11.26 MiB of inference assets.

For each question, the course supplies four solutions and a 1,024-dimensional representation of each solution. A verifier assigns scores to these candidates and returns one index. The evaluator checks the final numeric answer, including equivalent fractions and decimals. It does not check every reasoning step. [1]

The main question was whether learned interactions among the four candidates could improve on a simple linear verifier. The linear model already uses relative features, but has no learned attention between candidates. I expected the attention layer and different training objectives to produce useful differences in their rankings.

The starting model uses the supplied representations and 32 arithmetic and number-source features. The neural models read the same information, with an attention layer that compares candidates. One objective rewards assigning probability to any correct candidate; the other ranks correct candidates above incorrect ones. Their scores are combined after normalization within each question.

Data and evaluation

Split	Questions	Use
Training	4,755; 2,114 disagreement questions	Fit parameters and preprocessing
Validation	798; 355 disagreement questions	Early stopping and model selection
Test	467; all disagreement questions	Official score

A disagreement question has at least two distinct valid parsed answers among its candidates. It is the useful development subset because unanimous answers make selection much easier. The final CPU implementation scores 308/355 (86.76%) on this subset and 751/798 (94.11%) on full validation. The latter includes 443 unanimous groups, so its percentage is not directly comparable with the test result.

Only the supplied training split is used for fitting. All preprocessing statistics come from training disagreement questions. Validation was reused for several comparisons; its best scores should therefore be read as development results.

2. Models

Relative linear verifier

The baseline independently implements the relative-verifier design in [2]. For each question, it subtracts the mean candidate representation and divides by the average centered vector norm. The 32 scalar features are also centered within the question and scaled by their root mean square. Training-column means and scales then standardize the resulting 1,056 features.

Ten scalar features summarize checkable arithmetic, such as the fraction of incorrect equalities and whether the final answer appears in a calculation. The other 22 describe the answer and literal number sources: numbers found in the question or earlier calculations, new numbers, sign, size and denominator. These checks provide evidence about a solution without fully interpreting it.

The model fits candidate correctness with binary cross-entropy (BCE). Its L2 penalties are 0.05 for representation features and 0.003 for scalar features. Float64 L-BFGS runs for at most 350 iterations. Candidates with the same parsed answer receive a group score:

```
group score = highest logit in the group + 0.5 log(group size)
```

The count term gives repeated answers a modest advantage while allowing a strong minority answer to win. Six pairs of L2 penalties and count bonuses of 0, 0.5 and 1 were compared on validation.

Neural verifiers

Training statistics standardize the representations, followed by full-dimensional ZCA whitening with ridge 0.1. The network reads both this vector and its centered, normalized version within the question, together with the scalar features. Values are clipped to [-6, 6].

The network has eight members that share projection matrices and have their own input/output scales, following the efficient-ensemble idea in TabM [3]. Each member has width 128. One attention block compares the four candidates, with a learned same-answer bias, residual connections and layer normalization. Its feedforward block has dimensions 128-256-128. Candidate positions have no learned embeddings. Set attention is motivated by [4].

For scores s, the two objectives are:

```
Mass:     logsumexp(s over valid candidates)
        - logsumexp(s over correct candidates)
Pairwise: mean softplus(s_wrong - s_correct)
```

Both add 0.1 times valid-candidate BCE and average the loss over members. Training uses AdamW (learning rate 0.001, weight decay 0.03), dropout 0.25, batches of 128 questions, at most 150 epochs and patience 20. Seeds 17, 29 and 43 were compared. The submitted mass model uses seed 17, epoch 30; the pairwise model uses seed 29, epoch 38.

Score combination

Let L be the linear group scores, M the mass-model scores and P the pairwise scores. Z standardizes valid candidate scores within the current question, with a minimum standard deviation of 1e-6:

```
B = 0.5 Z(L) + 0.5 Z(M)
Final score = 0.5 Z(B) + 0.5 Z(P)
```

The neural logits are averaged over their eight members. The second normalization matters: this is not a direct 0.25L + 0.25M + 0.5P sum. The program chooses the highest final score and excludes invalid answers whenever a valid candidate exists.

3. What the comparisons showed

The recent experiments used the same training and validation splits throughout. Neural epochs were selected by validation disagreement accuracy, with validation loss breaking ties. The following counts use the saved development scores; neural scores in this comparison were computed on GPU. Rows show selected settings, not averages over seeds.

Method	Correct / 355	Accuracy
Relative linear verifier L	305	85.92%
Independent candidate network	308	86.76%
Joint mass model M	305	85.92%
Joint pairwise model P	305	85.92%
Joint answer-group objective	308	86.76%
Two-model combination B	309	87.04%
Final combination, development scores	309	87.04%
Final combination, deployed CPU	308	86.76%

The clearest gain came from combining L and M: removing either component reduced the count from 309 to 305. The two models made different mistakes, even though neither was the best standalone network. This ablation supports combining these components on this validation split.

The independent network reached 308, above the joint mass model's 305. The answer-group objective also reached 308. Adding the pairwise model tied the two-model combination rather than improving its validation count. Thus the attention models were not stronger on their own. Because seed and epoch choices also varied, these comparisons do not isolate the effect of attention.

CPU deployment

The final CPU run changed one validation choice and scored 308 rather than 309. The largest combined-score difference was 6.79e-7, enough to change a close decision. This was recorded before final test scoring. Weights and combination coefficients were kept unchanged. The report uses 308/355 for the actual CPU implementation.

Official test results

Fixed evaluated version	Correct / 467	Accuracy
Earlier Gaussian/path model	385	82.44%
Two-model combination B	396	84.80%
Submitted final combination	399	85.44%

The final combination was chosen from existing candidates tied at the best validation count, using model size and then name to break ties. Its weights and coefficients were fixed before its test evaluation. It answered three more test questions correctly than B, despite the validation tie.

Earlier test totals were already known during the project. These are records of successive fixed versions, not independent repetitions on a previously unseen test set. No individual test errors were inspected to design features, and the final score led to no further model changes.

4. Computation and reproduction

Inference

Measurement	Recorded value	Course limit
Prediction process	3.022 s	Included below
Independent scoring process	0.113 s	Included below
Complete fresh CPU evaluation	3.136 s	30 s
All inference assets, uncompressed	11,808,974 bytes (11.26 MiB)	16 MiB
Observed total threads	2, including the monitor	4

The local processor was a Xeon Platinum 8368Q. Timing includes process startup, imports, data and checkpoint loading, feature extraction and every model. CUDA was hidden; numerical libraries used one thread and CPU affinity was limited to four cores. The reference Xeon 8457C determines official timing eligibility.

Inference uses NumPy. The ten checkpoint files contain all weights, preprocessing statistics and configurations. Both stored ZCA transforms are included in the asset count. The official parser and scoring tools are unchanged.

Training cost

Recent experiment	Fits	Recorded stage time
Linear penalties	6	21.72 s
Independent / joint mass networks	3 / 3	56.33 / 55.85 s
Pairwise / answer-group networks	3 / 3	53.27 / 80.96 s

The two neural routes in each pair ran concurrently on separate RTX 4090 cards; each fit used one card. These paired times are not added as wall time. The course specifies CPU or one GPU no stronger than a 4090. The concurrent development schedule is disclosed here; it has not been confirmed as an allowed interpretation of that rule.

The submitted training command was also run serially on one exposed 4090. It completed these 18 fits in 274.07 seconds and reproduced all ten checkpoint files byte for byte, including all 62 arrays. This confirms the recorded environment, rather than promising identical training on every machine. The successful replay does not change the earlier concurrency disclosure.

These costs cover the recent model family. Earlier searches are recorded separately in the repository. Some historical records overlap, so they are not summed into a project total.

Reproduction files

The README gives exact installation, CPU evaluation and single-GPU training commands. Checkpoint evaluation needs no retraining. The official score JSON, resource measurements, seeds and file hashes are included, along with the record of training replay. The entry point remains `predict(bank, arrays, checkpoint)` and returns Python integer indices in input order. It reads the current inputs rather than stored predictions or question IDs.

5. Discussion

There is a practical reason to keep the linear verifier: it is cheap and makes different choices from the mass model. Evidence for the third branch is weaker. Compared with B, the final model increased assets from 5.64 to 11.26 MiB and CPU time from 2.696 to 3.136 seconds. Its three-answer test gain came without a validation gain. The extra branch fits the resource budget, but these results alone do not show that its cost would pay off on another bank.

What the model still misses

Arithmetic checks cannot establish that a solution used the right quantities. In an earlier review of training errors, a candidate correctly computed 8 x 5 = 40 total pieces of wood, then treated 40 as the number of pieces per item. Checking the multiplication does not detect that change in meaning. This example comes from an earlier model's training-error review, not a new error analysis of the final model's test set.

The scalar features match literal numbers and calculations; they do not reliably bind objects, units and times. This leaves room for a solution to make consistent calculations about the wrong quantity. Also, the outcome labels reward a correct final number even when some intermediate steps are faulty. The model's accuracy measures answer selection, not step-by-step mathematical verification.

Validation contains only 355 disagreement questions and was used for epoch, seed, penalty and combination choices. That limits confidence in small gains. A separate audit bank could reverse the ranking of these models. The results also do not establish that reading full reasoning text is better than using only final answers: that information-source comparison was not isolated in this model family.

Assistance and reuse

OpenAI Codex assisted method comparison, implementation, training orchestration, debugging and writing. The relative-verifier design follows [2], implemented independently; its published 394/467 total was known. Its weights, data and individual test predictions were not used. The networks use ideas from TabM and set attention [3,4], with all parameters trained from scratch on course data. No external data, additional pretrained weights or AI-generated semantic labels were used. Course parser/scorer code and GSM8K/Qwen attribution are retained.

References

[1] X. Wu. DASE7506 MP2: Selecting Reliable LLM Reasoning. Fall 2026. Official guide, accessed 31 October 2026.

[2] Anchmy. dase7506-mp2-group-relative-verifier, commit 08813bf6ad85fea5242fc40392f25dd8ba26e1cf.

[3] Y. Gorishniy, A. Kotelnikov and A. Babenko. TabM: Advancing Tabular Deep Learning with Parameter-Efficient Ensembling. ICLR 2025. arXiv:2410.24210.

[4] J. Lee et al. Set Transformer: A Framework for Attention-based Permutation-Invariant Neural Networks. ICML 2019, PMLR 97:3744-3753. Paper.